

Unit 4. Advanced Topics

4.0 Multi-Dimensional Arrays	309
4.1 Allocating Memory	324
4.2 String Class.....	337
4.3 Command Line.....	346
4.4 Instrumentation	356
Unit 4 Practice Test	361
Unit 4 Project : Sudoku	363

4.0 Multi-Dimensional Arrays

Sam had so much fun dabbling with ASCII-art that he thought he would try his hand at computer graphics. The easiest way to get started is to load an image from memory and display it on the screen. This seems challenging, however; memory (including the type of data stored in an array) is one-dimensional but images are two-dimensional. How can he store two-dimensional data in an array? How can he convert the one-dimensional data in a file into this array? While trying to figure this out, Sue introduces him to multi-dimensional arrays.

Objectives

By the end of this class, you will be able to:

- Declare a multi-dimensional array.
- Pass a multi-dimensional array to a function as a parameter.
- Read multi-dimensional data from a file and put it in an array.

Prerequisites

Before reading this section, please make sure you are able to:

- Declare an array to solve a problem (Chapter 3.0).
- Write a loop to traverse an array (Chapter 3.0).
- Pass an array to a function (Chapter 3.0)
- Write the code to read data from a file (Chapter 2.6).
- Write the code to write data to a file (Chapter 2.6).

Overview

Often we work with data that is inherently multi-dimensional. A few common examples include pictures (row and column), coordinates (latitude, longitude, and altitude), and position on a grid (x and y). The challenge arises when we need to store the multi-dimensional data in a memory store that is inherently one dimensional.

Consider the following code to put the numbers 0-15 on the screen:

```
for (int index = 0; index < 16; index++)
    cout << index << '\t';
cout << endl;
```

Observe how the numbers are one-dimensional (just an index). However, we would like to put the numbers in a nice two-dimensional grid that is 4×4 . How do we do this? The first step is we need some way to detect when we are on the 4th column. When we are on this column, we display a newline character rather than a white space to properly align the columns.

	column 0	column 1	column 2	column 3	
0	1	2	3		row 0
4	5	6	7		row 1
8	9	10	11		row 2
12	13	14	15		row 3

Are there any patterns in the numbers? Can we find any way to derive the row or column based on the index? The first thing to realize is that the column numbers seem to increase by one as the index increases by one. This occurs until we get to the end of the row. When that happens, the column number seems to reset.

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
column	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3

This pattern should be familiar. As we divide the index (index) by the number of columns (numCol), the remainder appears to be the column (column) value.

```
column = index % numCol;
```

The row value appears to be an entirely different equation. We increment the row value only after we increment four index values:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
row	0	0	0	0	1	1	1	1	2	2	2	2	3	3	3	3

This pattern is also familiar. We can derive row by performing integer division on index by numCol:

```
row = index / numCol;
```

Based on these observations, we can re-write our loop to display the first 16 whole numbers:

```
for (int index = 0; index < 16; index++)
    cout << index << (index % 4 == 3 ? '\n' : '\t');
```

In other words, when the column value (index % 4) is equal to the fourth column (== 3) then display a newline character ('\n') rather than the tab character ('\t'). We can re-write this more generally as:

```

/*****
 * DISPLAY NUMBERS
 * Display the first 'number' whole numbers
 * neatly divided into a grid of numCol columns
 *****/
void displayNumbers(int number, int numCol)
{
    for (int index = 0; index < number; index++)           // one dimensional index
        cout << index                                       // display the index
            << (index % numCol == numCol - 1 ? '\n' : '\t'); // break into rows
}

```

After converting an inherently one-dimensional value (index) into a two-dimensional pair (row & column), how do we convert two-dimensional values back into an index? To accomplish this, we need to recall the things we learned when going the other way:

- An increase in the index value yields an increase in the column value. To turn this around, we could also say that an increase in the column value yields an increase in the index value.
- The row value changes one fourth ($1 / \text{numCol}$) as often as the index value. To turn this around, a change in the row value yields a jump in the index value by four (numCol).

We can combine both these principles in a single equation:

```
index = row * 4 + column;
```

Check this equation for correctness:

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
row	0	0	0	0	1	1	1	1	2	2	2	2	3	3	3	3
column	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3

If we add one to the row, then the index jumps by four. If we add one to the column, the index jumps by one. Thus we have the ability to convert two-dimensional coordinates (row & column) into a one dimensional value (index). The general form of this equation (worth memorizing) is:

```
index = row * numCol + column;
```

Why would we ever want to do this? Consider the scenario when we want to put the multiplication tables for the values 0 through 3 in an array. This can be accomplished with:

```
{
    int grid[4 * 4];                // the area of the array is the width
                                   // times the height.
    for (int row = 0; row < 4; row++) // rows first, 0...3
        for (int col = 0; col < 4; col++) // columns next, also 0...3
            grid[row * 4 + col] = row * col; // convert to row,col to index for the []
}
```

In memory, the resulting grid array appears as the following:

	row 0				row 1				row 2				row 3			
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
grid	0	0	0	0	0	1	2	3	0	2	4	6	0	3	6	9

In both of these cases (converting 2-dimensional to 1 and converting 1-dimensional to 2), the same piece of information is needed: the number of columns (numCol) in the data. This should make sense. If you have 32 items in a data-set, is the grid 1×32 or 2×16 or 4×8 or 8×4 or 16×2 or 32×1? Each of these possibilities is equally likely. One must know either the number of columns or the number of rows to make the conversion.

Syntax

As you may have noticed, multi-dimensional arrays are quite commonly needed to solve programming problems. Similarly, the conversion from index to coordinates and back is tedious and overly complicated. Fortunately, there is an easier way:

Declaring an array	Referencing an array	Passing as a parameter
Syntax: <pre><Type> <name>[size][size]</pre> Example: <pre>int data[200][15];</pre> A few details: • Any data-type can be used. • The size must be a natural number {1, 2, etc.} and not a variable.	Syntax: <pre><name>[index][index]</pre> Example: <pre>cout << data[i][j];</pre> A few details: • The index starts with 0 and must be within the valid range.	Syntax: <pre>(<Type> <name>[][size])</pre> Example: <pre>void func(int data[][15])</pre> A few details: • You must specify the base-type. • No size is passed in the square brackets [].

Declaring an array

Multi-dimensional arrays are declared by specifying the base-type and the size of each dimension. The basic syntax is:

```
<base-type> <variable>[<number of rows>][<number of columns>;
```

A grid of integers that is 3×4 can be declared as:

```
int grid[4][3];
```

We can also initialize a multi-dimensional array at declaration time. The best way to think of the initialization syntax is “an array of arrays.” Consider the following example:

```
{  
    int grid[4][3] =  
    {  
        { 8, 12, -5 }, // row 0  
        { 421, 4, 153 }, // row 1  
        { -15, 20, 91 }, // row 2  
        { 4, -15, 182 }, // row 3  
    };  
}
```



Sue's Tips

Notice how the horizontal dimension comes *second* in multi-dimensional arrays. In Geometry, we learned to specify coordinates as (X, Y) where the horizontal dimension comes first. Multi-dimensional arrays are the opposite! Rather than trying to re-learn (Y, X) (which just doesn't feel right, does it?), it is more convenient to use (Row, Column) as our array dimensions.

Storing a digital image is a slightly more complex example. Each pixel consists of three values (red, green, and blue) with 256 possible values in each (char). The pixels themselves are arrayed in a two-dimensional image ($4,000 \times 3,000$). The resulting declaration is:

```
char image[3][3000][4000];
```

In this example, each element is a char (eight bits in a byte so there are 2^8 possible values). The first dimension ([3]) is for the three channels (red, green, and blue). The next is the horizontal size of the image (4,000). The final dimension is the vertical dimension (3,000). The total size of the image is:

```
int size = sizeof(char) * sizeof(3) * sizeof(3000) * sizeof(4000);
```

This is 36,000,000 bytes of data (34.33 megabytes). A twelve mega-pixel image is rather large!

Referencing an array

When referencing an array, it is important to specify each of the dimensions. Again, we use the vertical dimension first so we use (Row, Column) variables rather than (X, Y). Back to our 3×4 grid example:

```
{
    int grid[4][3] =
    {
        { 8, 12, -5 }, // row 0
        { 421, 4, 153 }, // row 1
        { -15, 20, 91 }, // row 2
        { 4, -15, 182 }, // row 3
    };

    int row;    // vertical dimension
    int col;    // horizontal dimension

    cout << "Specify the coordinates (X, Y) "; // people think in terms of X,Y
    cin >> col >> row;

    assert(row >= 0 && row < 4);                // a loop would be a better tool here
    assert(col >= 0 && col < 3);                // always check before indexing into
                                                // an array

    cout << grid[row][col] << endl;
}
```

Working with more than two-dimensions is the same. Back to our image example consisting of a two-dimensional grid of pixels ($4,000 \times 3,000$) where each pixel has three values. If the user wishes to find the value of the top-left pixel, then the following code would be required:

```
cout << "red:  " << image[0][0][0] << endl
     << "green: " << image[1][0][0] << endl
     << "blue:  " << image[2][0][0] << endl;
```

Passing as a parameter

Passing arrays as parameters works much the same for multi-dimensional arrays as they do for their single-dimensional brethren. There is one important exception, however. Recall from earlier in the semester (Chapter 3.0) that arrays are just pointers to the first item in the list. Only having this pointer, the callee does not know the length of the buffer. For this reason, it is important to pass the size of the array as a parameter.

There is another important component to understanding multi-dimensional array parameters. Recall that, for a given 32 slots in memory, there may be many possible ways to convert it into a two-dimensional grid { (32×1) , (16×2) , (8×4) , (4×8) , (2×16) , or (1×32) }. The only way to know which conversion is correct is to know the number of columns (typically called the numCol variable). This information is essential to performing the conversion.

When using the double-square-bracket notation for multi-dimensional arrays (array[3][4] instead of array[3 * numCol + 4]), the compiler needs to know the numCol value. The same is true when passing multi-dimensional

arrays as parameters. In this case, we specify the size of all the dimensions except the left-most dimension. Back to our 3×4 example, a prototype might be:

```
void displayGrid(int array[][3]);    // column size must be present
```

Back to our image example, the following code will fill the image with data from a file.

```

/*****
 * READ IMAGE
 * Read the image data from a file
 *****/
bool readImage(unsigned char image[][3000][4000],    // specify all dimensions but first
               const char fileName[])               // also need the filename as const
{
    // open stream
    ifstream fin(fileName);
    if (fin.fail())                                // never forget error checking
        return false;                             // return and report

    bool success = true;                           // our return value

    // read the grid of data
    for (int row = 0; row < 3000; row++)            // rows are always first
        for (int col = 0; col < 4000; col++)        // then columns
            for (int color = 0; color < 3; color++) // three color dimensions: r, g, b
            {
                int input;                          // data in the file is a number so
                fin >> input;                        // we read it as an integer
                if ( input < 0 || input >= 256 ||     // before storing it as a
                    fin.fail())                     // char (a small integer). Make
                    success = false;                // sure it is valid!
                image[color][row][col] = input;
            }

    // paranoia!
    if (fin.fail())                                // report if anything bad happened
        success = false;

    // make like a tree
    fin.close();                                    // never forget to close the file
    return success;
}

/*****
 * MAIN
 * Simple driver for readImage
 *****/
int main()
{
    unsigned char image[3][3000][4000];            // 12 megapixel image
    if (!readImage(image, "image.bmp"))             // .bmp images are just arrays
        return 1;                                  // of pixels! Note that they
    else                                            // are binary files, not text
        return 0;                                  // files so this will not quite
                                                    // work the way you expect...
}

```

One quick disclaimer about the above example... Images are stored not as text files (which can be opened and read in emacs) but as binary files (such as a.out. Try opening it in emacs to see what I mean). To make this above example work, we will need to create 36 million integers ($3 \times 3,000 \times 4,000$), each of which with a value between 0 and 255. That might take a bit of patience.

Example 4.0 – Array of Strings

Demo

This example will demonstrate how to create, pass to a function, and manipulate an array of strings. This is a multi-dimensional array of characters.

Solution

Since strings are arrays, to have an array of strings we will need a two dimensional array. Note that the first dimension must be the number of strings and the second the size of each.

```

/*****
 * PROMPT NAMES
 * Prompt the user for his or her name
 *****/
void promptNames(char names[][256])    // the column dimension must be the
{                                     //      buffer size
    // prompt for name (first, middle, last)
    cout << "What is your first name? ";
    cin  >> names[0];                  // passing one instance of the array
    cout << "What is your middle name? "; //      of names to the function CIN
    cin  >> names[1];                  // Note that the data type is
    cout << "What is your last name? ";  //      a pointer to a character,
    cin  >> names[2];                  //      what CIN expects
}

/*****
 * MAIN
 * Just a silly demo program
 *****/
int main()
{
    char names[3][256];                // arrays of strings are multi-
                                     //      dimensional arrays of chars

    // fill the array
    promptNames(names);                // pass the entire array of strings

    // first name:
    cout << names[0] << endl;          // this is an array of characters

    // middle initial
    cout << names[1][0] << endl;       // first letter of second string

    // loop through the names for output
    for (int i = 0; i < 3; i++)
        cout << names[i] << endl;

    return 0;
}

```

Challenge

As a challenge, extend the names array to include an individual's title. Thus the promptNames() function will consider the fourth row in the names array to be the title. You will also need to modify main() so the output can be displayed.

See Also

The complete solution is available at [4-0-arrayOfStrings.cpp](#) or:

/home/cs124/examples/4-0-arrayOfStrings.cpp



Example 4.0 – Array of Integers

Demo

This example will create a 4×4 array of integers. This will be done both the old-fashioned way of using a single-dimensional array as well as the new double-bracket notation. In both cases, the arrays will be filled with multiplication tables ($\text{row} * \text{col}$).

The 16 items in a 4×4 multiplication table represented as a single-dimensional array are:

	row 0				row 1				row 2				row 3			
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
grid	0	0	0	0	0	1	2	3	0	2	4	6	0	3	6	9

To write a function to fill this array, two parameters are needed: the number of rows (`numRow`) and the number of columns (`numCol`).

```
void fillArray1D(int grid[], int numCol, int numRows)
{
    for (int row = 0; row < numRows; row++)
        for (int col = 0; col < numCol; col++)
            grid[row * numCol + col] = row * col;
}
```

To do the same thing as a multi-dimensional array, the data representation is:

	grid[0][]				grid[1][]				grid[2][]				grid[3][]			
	0,0	0,1	0,2	0,3	1,0	1,1	1,2	1,3	2,0	2,1	2,2	2,3	3,0	3,1	3,2	3,3
grid	0	0	0	0	0	1	2	3	0	2	4	6	0	3	6	9

To work with multi-dimensional arrays, the compiler has to know the number of rows in the array. This means that, unlike with the single-dimensional version, we can only pass the `numRow` parameter. The `numCol` must be an integer literal specified in the parameter.

```
void fillArray2D(int grid[][4], int numRows)
{
    for (int row = 0; row < numRows; row++)
        for (int col = 0; col < 4; col++)
            grid[row][col] = row * col;
}
```

Challenge

As a challenge, change the program to display a 5×6 table. What needs to change in the calling function? What needs to change in the two fill functions?

See Also

The complete solution is available at [4-0-arrayOfIntegers.cpp](#) or:

`/home/cs124/examples/4-0-arrayOfIntegers.cpp`



Example 4.0 – Convert Place To Points

Demo

Recall that there are two main uses for arrays: either they are a “bucket of variables” useful for storing lists of items, or they are tables useful for table-lookup scenarios. This example will demonstrate the table-lookup use for arrays.

Problem

Sam can make varsity on the track team if he gets 12 points. There are 10 races and points are awarded according to his placing:

Place	Points
1	5
2	3
3	2
4	1

Solution

We can create a data-driven program to compute how many points Sam will get during the season. If he gets 12 points and his varsity jacket, possibly Sue will want to go on another date with him!

```
{
    int points = 0;                                // initial points for the season
    int breakdown[4][2] =
    {
        {1, 5},                                     // 1st place gets 5 points
        {2, 3},                                     // 2nd place gets 3...
        {3, 2},
        {4, 1}
    };

    // Loop through the 10 races in the season
    for (int cRace = 0; cRace < 10; cRace++)          // “cRace” for “count Race”
    {
        // get the place for a given race
        int place;
        cout << "what was your place? ";
        cin >> place;

        // add the points to the total
        for (int cPlace = 0; cPlace < 4; cPlace++) // Loop through all the places
            if (breakdown[cPlace][0] == place)    // if place in the table matches
                points += breakdown[cPlace][1];    // assign the points
    }

    cout << points << endl;
}
```

Observe how the first column is directly related to the row (`breakdown[row][0] == row + 1`). This means we technically do not need to have a multi-dimensional array for this problem.

Challenge

As a challenge, adapt this solution to the points awarded to the finishers at the Tour de France:

20	17	15	13	12	10	9	8	7	6	5	4	3	2	1
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---

In other words, the first finisher wins 20 points, the second 17, and so on.

See Also

The complete solution is available at [4-0-convertPlaceToPointers.cpp](#) or:

`/home/cs124/examples/4-0-convertPlaceToPoints.cpp`



Example 4.0 – Read Scores

Demo

This example will demonstrate how to fill a multi-dimensional array of numbers from a file, how to display the contents of the array, and how to process data from the array.

Problem

Write a program to read assignment scores from 10 students, each student completing 5 assignments. The program will then display the average score for each student and for each assignment. If there were three students, the file containing the scores might be:

```
92 87 100 84 95
71 79 85 62 81
95 100 100 92 99
```

Solution

The function to read five scores for numStudents individuals is the following:

```
bool readData(int grades[][5], int numStudents, const char * fileName)
{
    ifstream fin(fileName);
    if (fin.fail())
        return false;

    // read the data from the file, one row (student) at a time
    for (int iStudent = 0; iStudent < numStudents; iStudent++)
    {
        // read all the data for a given student: 5 assignments
        for (int iAssign = 0; iAssign < 5; iAssign++)
            fin >> grades[iStudent][iAssign];

        if (fin.fail())
        {
            fin.close();
            return false;
        }
    }

    fin.close();
    return true;
}
```

Observe how two loops are required: the outer loop `iStudent` to go through all the students in the list. The inner loop `iAssign` reads all the scores for a given student.

Challenge

As a challenge, modify the above program and the associated data file to contain 6 scores for each student. What needs to change? Can you create a `#define` to make changes like this easier?

See Also

The complete solution is available at [4-0-readScores.cpp](#) or:

```
/home/cs124/examples/4-0-readScores.cpp
```



Example 4.0 – Pascal’s Triangle

Pascal’s triangle is a triangular array of numbers where each value is the sum of the two numbers “above” it:

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
```

Consider the number 6 in the second from bottom row. It is the sum of the 3 and the 3 from the preceding row. For a graphical representation of this relationship, please see [this animation](#).

We will implement Pascal’s triangle by turning the triangle on its side:

1	1	1	1	1	1	1	1	1	1	1
1	2	3	4	5	6	7	8	9	10	11
1	3	6	10	15	21	28	36	45	55	66
1	4	10	20	35	56	84	120	165	220	286
1	5	15	35	70	126	210	330	495	715	1001
1	6	21	56	126	252	462	792	1287	2002	3003
1	7	28	84	210	462	924	1716	3003	5005	8008
1	8	36	120	330	792	1716	3432	6435	11440	19448
1	9	45	165	495	1287	3003	6435	12870	24310	43758
1	10	55	220	715	2002	5005	11440	24310	48620	92378
1	11	66	286	1001	3003	8008	19448	43758	92378	184756

One the first row, the values are 1, 1, 1, etc. From here, the first item on each new row is also the value 1. Every other item is the sum of the previous row and the previous column.

```
void fill(int grid[][SIZE])
{
    // 1. fill the first row
    for (int column = 0; column < SIZE; column++)
        grid[0][column] = 1;

    for (int row = 1; row < SIZE; row++)
    {
        // 2. The first item on a new row is 1
        grid[row][0] = 1;

        // 3. Every other item is the sum of the item above and to the left
        for (int column = 1; column < SIZE; column++)
            grid[row][column] = grid[row - 1][column] + grid[row][column - 1];
    }
}
```

The complete solution is available at [4-0-pascalsTriangle.cpp](#) or:

/home/cs124/examples/4-0-pascalsTriangle.cpp



Problem 1

What is returned if the input is 82?

```
char convert(int input)
{
    char letters[] = "ABCDF";
    int minRange[] =
        {90, 80, 70, 60, 0};

    for (int i = 0; i < 5; i++)
        if (minRange[i] <= input)
            return letters[i];

    return 'F';
}
```

Answer:

Please see page 218 for a hint.

Problem 2

What is the output of the following code?

```
int num(int n, float * a)
{
    int s = 0;

    for (int i = 0; i < n; i++)
        s += (a[i] >= 80.0);

    return s;
}

int main()
{
    cout << num(5, {71.3, 84.7, 63.9, 99.8, 70})
        << endl;

    return 0;
}
```

Answer:

Please see page 223 for a hint.

Problem 3

What is the output of the following code?

```
{
    char a[8] = "Rexburg";
    bool b[8] =
        {true,  false, true,  true,
         false, true,  true,  false};

    for (int i = 0; i < 8; i++)
        if (b[i])
            cout << a[i];

    cout << endl;
}
```

Answer:

Please see page 218 for a hint.

Problem 4

What is the syntax error?

```
{
    char letter = 'a';

    switch (letter)
    {
        case 'a':
            cout << "A\n";
        case true:
            cout << "B!\n";
            break;
            break;
        case 1:
            cout << "C!\n";
            break;
    }
}
```

Answer:

Please see page 287 for a hint.

Problem 5

Declare a variable to represent a Sudoku board:

Please see page 49 for a hint.

Problem 6

What is wrong with each of the following array declarations?

```
int x = 6;  
int array[x][x];
```

```
const float x = 1;  
int array[x]
```

```
int array[][]
```

```
int array[6 * 5 + 2][4 / 2];
```

Please see page 215 for a hint.

Problem 7

What is the output of the following code fragment?

```
{  
    int array[2][2] =  
        { {3, 4}, {1, 2} };  
  
    cout << array[1][0];  
}
```

Answer:

Please see page 313 for a hint.

Problem 8

Consider an array that is declared with the following code:

```
int array[7][21];
```

Write a prototype of a function that will accept this array as a parameter.

Answer:

Please see page 313 for a hint.

Assignment 4.0

Write a function to read a Tic-Tac-Toe board into an array. The file format is:

```
x o .  
.:.  
.x.
```

The character 'x' means that the 'x' player has taken that square. The character '.' means that the square is currently unclaimed. There is one space between each symbol.

Note: You will need to store the results in a 2D array. The function should take the filename as a parameter.

Write a function to display a Tic-Tac-Toe board on the screen. Given the above board, the output is:

```
x | o |  
---+---  
| | |  
---+---  
| x |
```

Write a function to write the board to a file. The file format is the same as with the read function.

Example

The user input is underlined.

```
Enter source filename: board.txt  
x | o |  
---+---  
| | |  
---+---  
| x |  
Enter destination filename: board2.txt  
File written
```

Assignment

The test bed is available at:

```
testBed cs124/assign40 assignment40.cpp
```

Don't forget to submit your assignment with the name "Assignment 40" in the header.

Please see page 234 for a hint.